

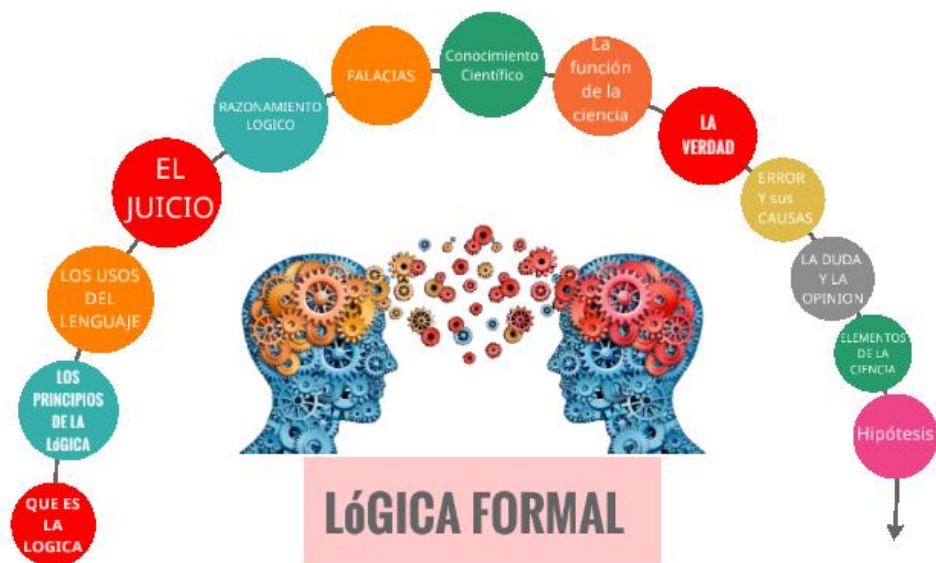
Nombre: Luis Alberto Vargas González

Fecha: 25/08/2025

Actividad y unidad: U5 A3. Portafolio de Evidencias.

Clase: Razonamiento y Representación del conocimiento.

Maestría en Ciberseguridad.



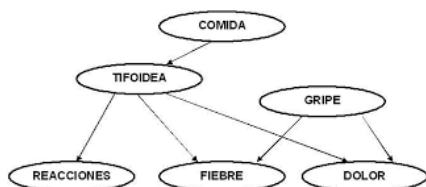
INDICE.

1. Introducción.
2. Ejercicios U2 A2.
3. Ejercicios U3 A2.
4. Ejercicios U4 A2.
5. Ejercicios U5A2.
6. Conclusión general.
7. Referencias Bibliográficas.

Introducción.

En esta última actividad se describirá de manera breve y concreta las actividades que se llevaron a cabo en el curso de Razonamiento y Representación del conocimiento en donde pudimos estudiar de manera básica, la programación lógica en sus diversos lenguajes como PROLOG, ASP, la introducción a la planificación con el método de STRIPS, muy útil si queremos enfocarnos en áreas de IA y Ciberseguridad. Adicionalmente estudiamos un poco sobre la web semántica y sus ontologías, así como una serie de prácticas que llevamos a cabo en donde por medio de objetos, hechos y relaciones pudimos crear gráficos de conocimiento, lo cual es útil en Ciberseguridad para poder construir modelos de conocimiento acerca de cómo operan cierto tipo de malware, con que objetos, plataformas y programas interacciona para poder activarse. Además, por último, pudimos estudiar el tema de redes Bayesianas que es una forma de poder construir y calcular probabilidades dentro de muchas áreas del conocimiento, en ciberseguridad nos es útil para poder estimar probabilidades de ataques, intrusiones a redes corporativas, estimar probabilidades de infección de software y hardware.

Es importante recalcar que este portafolio es una colección de los desarrollos que se tuvieron dentro de la materia, y se presenta para que el lector interesado pueda tener una noción de lo que estudiamos dentro de esta materia de manera breve y concisa, y que pueda además investigar por su cuenta estas actividades y temas y pueda obtener por medio de este documento, una idea general de como conectar los temas entre sí y como obtener una guía introductoria a este fascinante mundo de la lógica , la programación lógica , redes bayesianas, web semántica y planificación.



Ejercicios U2 A2.

1.

```
1, % Hechos: temperaturas por ciudad
```

```
temp(cdmx, 24).
```

```
temp(puebla, 24).
```

```
temp(leon, 25).
```

```
temp(san_luis_potosi, 27).
```

```
temp(aguascalientes, 25).
```

```
temp(guadalajara, 25).
```

```
temp(monterrey, 36).
```

```
temp(ciudad_juarez, 29).
```

```
% Reglas para clasificar el clima
```

```
templado(C) :- temp(C, T), T < 25.
```

```
calido(C) :- temp(C, T), T > 25.
```

```
% Mostrar solo las ciudades clasificadas
```

```
#show templado/1.
```

```
#show calido/1.
```

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  ASP
clingo clima.lp
clingo version 5.8.0
Reading from clima.lp
Solving...
Answer: 1 (Time: 0.001s)
calido(san_luis_potosi) calido(monterrey) calido(ciudad_juarez) templado(cdmx) templado(puebla)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.001s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s
```

2. A) Se lee como: "X es hijo de y si Y es padre de X".

B) % Hechos

```
padre(ana, juan).
padre(juan, karina).
padre(juan, dany).

% Regla para deducir hijo.

hijo(X,Y) :- padre(Y,X).

% Mostrar la relación hijo.

#show hijo/2.
```

```
clingo familia.lp

clingo version 5.8.0
Reading from familia.lp
Solving...
Answer: 1 (Time: 0.001s)
hijo(juan,ana) hijo(karina,juan) hijo(dany,juan)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.001s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.000s
```

```
#show p/2.
```

```
clingo version 5.8.0
Reading from aritmetica_a.lp
Solving...
Answer: 1 (Time: 0.001s)
p(0,41) p(1,43) p(2,47) p(3,53) p(4,61) p(5,71) p(6,83) p(7,97) p(8,113) p(9,131) p(10,151)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.001s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s
```

B). $p(N, N*N + N + 41) :- N+1 = 1..11$.

```
#show p/2.
```

```
clingo version 5.8.0
Reading from aritmetica_b.lp
Solving...
Answer: 1 (Time: 0.001s)
p(0,41) p(1,43) p(2,47) p(3,53) p(4,61) p(5,71) p(6,83) p(7,97) p(8,113) p(9,131) p(10,151)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.001s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s
```

~/E/L/Unidad 2/aritmetica_3 30 x at 12:42:09

C). $p(N, N*N + N + 41) :- N = -10..10, N \geq 0$.

```
#show p/2.
```

B). $p(N, N*N + N + 41) :- N+1 = 1..11.$

#show p/2.

```
clingo version 5.8.0
Reading from aritmética_b.lp
Solving...
Answer: 1 (Time: 0.001s)
p(0,41) p(1,43) p(2,47) p(3,53) p(4,61) p(5,71) p(6,83) p(7,97) p(8,113) p(9,131) p(10,151)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.001s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s
```

C). $p(N, N*N + N + 41) :- N = -10..10, N >= 0.$

#show p/2.

```
clingo aritmética_c.lp
clingo version 5.8.0
Reading from aritmética_c.lp
Solving...
Answer: 1 (Time: 0.002s)
p(0,41) p(1,43) p(2,47) p(3,53) p(4,61) p(5,71) p(6,83) p(7,97) p(8,113) p(9,131) p(10,151)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.002s
```

D). $p(M, N) :- N = 1..4, N = 2*M.$

#show p/2.

D). $p(M, N) :- N = 1..4, N = 2 * M.$

$\#show\ p/2.$

```
~/E/L/Unidad 2/aritmética 3 65 x at 12:47:02
$ clingo aritmética_d.lp
clingo version 5.8.0
Reading from aritmética_d.lp
Solving...
Answer: 1 (Time: 0.002s)
p(1,2) p(2,4)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.002s
```


4. A) $p(N, (-1)^{**N}) :- N = 0..4$.

#show p/2.

```
~/E/Logica y representacion conoc./Unidad 2/arimetica_4 at 01:11:15
clingo arimetica_a.lp
clingo version 5.8.0
Reading from arimetica_a.lp
Solving...
Answer: 1 (Time: 0.002s)
p(0,1) p(1,-1) p(2,1) p(3,-1) p(4,1)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.002s
```

B). $p(N, M) :- N = 1..4, M = 1..N$.

#show p/2.

```
~/E/Logica y representacion conoc./Unidad 2/arimetica_4 1 x at 01:13:43
clingo arimetica_b.lp
}clingo version 5.8.0
Reading from arimetica_b.lp
Solving...
Answer: 1 (Time: 0.001s)
p(1,1) p(2,1) p(2,2) p(3,1) p(3,2) p(3,3) p(4,1) p(4,2) p(4,3) p(4,4)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s
```

5.

% Parámetros simbólicos (asignados como hechos)

a(1).

b(1).

c(41).

n(10).

% Regla que aplica el polinomio

p(N, A*N*N + B*N + C) :-

a(A), b(B), c(C), n(L), N = 0..L.

#show p/2.

```
clingo arimetica_5.lp
clingo version 5.8.0
Reading from arimetica_5.lp
Solving...
Answer: 1 (Time: 0.003s)
p(0,41) p(1,43) p(2,47) p(3,53) p(4,61) p(5,71) p(6,83) p(7,97) p(8,113) p(9,131) p(10,151)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.003s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.002s
```

6.

```
father(juan, maria).  
mother(pedro, ana).  
  
parent(X,Y) :- father(X,Y).  
parent(X,Y) :- mother(X,Y).  
  
#show parent/2.
```

```
~/Escritorio/Logica y Representacion Conoc./Unidad 2  
clingo parent.lp  
  
clingo version 5.8.0  
Reading from parent.lp  
Solving...  
Answer: 1 (Time: 0.002s)  
parent(pedro,ana) parent(juan,maria)  
SATISFIABLE  
  
Models      : 1  
Calls       : 1  
Time        : 0.003s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)  
CPU Time    : 0.002s
```

7.

`n(20)` % Límite superior para N.

% Un número N es compuesto si hay un divisor I entre 2 y N-1

`compuesto(N) :- n(L), N = 2..L, I = 2..N-1, N \ I == 0.`

% Un número N es primo si no es compuesto

`prime(N) :- n(L), N = 2..L, not compuesto(N).`

`#show prime/1.`

```
~/Escritorio/Logica y Representacion Conoc./Unidad 2
clingo primos.lp
clingo version 5.8.0
Reading from primos.lp
Solving...
Answer: 1 (Time: 0.004s)
prime(2) prime(3) prime(5) prime(7) prime(11) prime(13) prime(17) prime(19)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.004s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s
```

8.

`n 5` % Cambia esto para calcular hasta ota cantidad de numeros

```
fac(0, 1).
fac(N1, F1) :-
fac(N, F),
n(Max),
N < Max,
N1 = N + 1,
F1 = F * N1.

% Mostrar solo los resultados finales (F)
fac_result(F) :- fac(_, F).

#show fac_result/1.
```

```
~/Escritorio/Logica y representacion conoc./Unidad 2
clingo factorial.lp
clingo version 5.8.0
Reading from factorial.lp
Solving...
Answer: 1 (Time: 0.002s)
fac_result(1) fac_result(2) fac_result(6) fac_result(24) fac_result(120)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.002s
```

9. A). 1 (p 1 10).

```
#show p/1.
```

```
~/E/Logica y representacion conoc./Unidad 2/ejercicio 9 at 01:55:28
clingo eleccion_a.lp --models=5

clingo version 5.8.0
Reading from eleccion_a.lp
Solving...
Answer: 1 (Time: 0.001s)
p(10)
Answer: 2 (Time: 0.001s)
p(3)
Answer: 3 (Time: 0.001s)
p(3) p(10)
Answer: 4 (Time: 0.001s)
p(4)
Answer: 5 (Time: 0.002s)
p(3) p(4)
SATISFIABLE

Models      : 5+
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s
```

B).

{ elected(ann; bob; carol; dan; elaine; fred) } = 3.

#show elected/1.

```
~/E/Logica y representacion conoc./Unidad 2/ejercicio_9 10 x at 01:55:39
clingo eleccion_b.lp --models=3

clingo version 5.8.0
Reading from eleccion_b.lp
Solving...
Answer: 1 (Time: 0.002s)
elected(fred) elected(elaine) elected(carol)
Answer: 2 (Time: 0.002s)
elected(fred) elected(elaine) elected(bob)
Answer: 3 (Time: 0.002s)
elected(fred) elected(carol) elected(bob)
SATISFIABLE

Models      : 3+
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.002s
```

10. A).

```
{ p q } = 1.
```

```
#show p/0.
```

```
#show q/0.
```

```
~/Escritorio/Logica y representacion conoc./Unidad 2
clingo disyuncion.lp --models=2

clingo version 5.8.0
Reading from disyuncion.lp
Solving...
Answer: 1 (Time: 0.003s)
p
Answer: 2 (Time: 0.003s)
q
SATISFIABLE

Models      : 2
Calls       : 1
Time        : 0.003s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s

~/Escritorio/Logica y representacion conoc./Unidad 2
```


B). $p(a); q(b).$

$:- \text{not } p(a), \text{not } q(b).$

```
~/Escritorio/Logica y representacion conoc./Unidad 2 at 02:10:54
clingo restriccion.lp --models=3

clingo version 5.8.0
Reading from restriccion.lp
Solving...
Answer: 1 (Time: 0.001s)
q(b)
Answer: 2 (Time: 0.001s)
p(a)
SATISFIABLE

Models      : 2
Calls       : 1
Time        : 0.001s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.001s

~/Escritorio/Logica y representacion conoc./Unidad 2 30 x at 02:11:19
```

11. A)

```
{p(1..10)}.
```

```
:- p(2*X).
```

```
~/Escritorio/Logica y representacion conoc./Unidad 2 10 x at 02:13:55
clingo par_rest.lp --models=3

clingo version 5.8.0
Reading from par_rest.lp
Solving...
Answer: 1 (Time: 0.002s)

Answer: 2 (Time: 0.002s)
p(3)
Answer: 3 (Time: 0.002s)
p(5)
SATISFIABLE

Models      : 3+
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.002s

~/Escritorio/Logica y representacion conoc./Unidad 2 10 x at 02:14:09
clingo par_rest.lp --models=6

clingo version 5.8.0
Reading from par_rest.lp
Solving...
Answer: 1 (Time: 0.002s)

Answer: 2 (Time: 0.002s)
p(3)
Answer: 3 (Time: 0.002s)
p(5)
Answer: 4 (Time: 0.002s)
p(3) p(5)
Answer: 5 (Time: 0.002s)
p(7)
Answer: 6 (Time: 0.002s)
p(5) p(7)
SATISFIABLE
```

B)

B) $p(1,1).$

$q(x) :- X = 1..2, \text{not } p(X, _).$

$\#show\ q/1.$

```
clingo not_exist.lp
clingo version 5.8.0
Reading from not_exist.lp
Solving...
Answer: 1 (Time: 0.001s)
q(2)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.002s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.002s
```

Proyecto.

```

% Posiciones del tablero (1 a 3 en filas y columnas)

fila(1..3).

col(1..3).

% Jugadas hechas (puedes editar estas para probar)

jugada(1,1,x).

jugada(1,2,x).

jugada(1,3,x).

jugada(2,1,o).

jugada(2,2,o).

jugada(3,1,o).

% Regla: gana X si hay 3 en fila

gana(x) :- jugada(R,1,x), jugada(R,2,x), jugada(R,3,x), fila(R).

% Regla: gana O si hay 3 en fila

gana(o) :- jugada(R,1,o), jugada(R,2,o), jugada(R,3,o), fila(R).% Regla: gana por columna

gana(x) :- jugada(1,C,x), jugada(2,C,x), jugada(3,C,x), col(C).

gana(o) :- jugada(1,C,o), jugada(2,C,o), jugada(3,C,o), col(C).

% Regla: gana por diagonal

gana(x) :- jugada(1,1,x), jugada(2,2,x), jugada(3,3,x).

gana(x) :- jugada(1,3,x), jugada(2,2,x), jugada(3,1,x).

gana(o) :- jugada(1,1,o), jugada(2,2,o), jugada(3,3,o).

gana(o) :- jugada(1,3,o), jugada(2,2,o), jugada(3,1,o).

#show gana/1.

```

```
❌ clingo gato.lp
clingo version 5.8.0
Reading from gato.lp
Solving...
Answer: 1 (Time: 0.003s)
gana(x)
SATISFIABLE

Models      : 1
Calls       : 1
Time        : 0.003s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.003s
```

~/Escritorio/Logica y representacion conoc /Unidad 2 30 x at 02:24:45

Conclusión UI2 A2: Como pudimos observar tenemos aquí las respuestas de los problemas que se nos presentaron en la unidad y en la actividad 2, siendo estos últimos resueltos mediante ASP-Clingo , como pudimos ver, la mayoría de los problemas se tratan de validar las fórmulas que previamente nos dieron en la tarea, ejecutarlas y generar un resultado a partir de ellas , con lo cual observamos que muchas de ellas simplemente lo que tenía que hacerse era copia la formula o problema, adaptarlo brevemente al lenguaje de Clingo, generando sus respectivas variables, con sus respectivos resultados.

Unidad 3 A2.

Ejercicio. Formalizar el dominio de satélites en STRIPS. Se hará en cuatro fases:

1. Identificar los predicados (cómo representamos el estado del mundo).
 2. Definir las acciones con sus precondiciones, añadidos y borrados.
 3. Ejemplo de estado inicial y meta.
 4. Ejemplo de plan.
 5. Predicados STRIPS Los predicados describen condiciones del mundo. A partir del ejercicio, los predicados relevantes serían:
 6. apuntando(sat, obj) → El satélite sat está apuntando al objetivo obj.
 7. instrumento (sat, inst, tipo) → El satélite sat tiene un instrumento inst del tipo tipo (ej. camara_alta, camara_baja, medidor_A, medidor_B, medidor_C).
 8. calibrado (sat, inst) → El instrumento inst del satélite sat está calibrado.
 9. imagen_tomada(obj, resol) → Se ha tomado la imagen del objetivo obj con resolución resol (alta, media o baja).
 10. medicion_tomada(obj, tipo) → Se ha hecho la medición del objetivo obj para la señal de tipo tipo. Además, podríamos necesitar algunos predicados auxiliares:
 11. resolucion(inst, tipo_resol) → Solo para cámaras, indica qué resoluciones puede manejar el instrumento.
 12. objetivo_tipo(obj, tipo) → Indica si el objetivo requiere una imagen o medición y de qué tipo.
2. Definición de Acciones STRIPS Ahora formalizamos las acciones como (Nombre / PRE / ADD / DEL):

2.1. Apuntar Satélite • Acción: apuntar (sat, obj) • Precondiciones: o (ninguna extra si asumimos que siempre puede girar) • Añadidos: o apuntando (sat, obj) • Borrados: o (opcional) apuntando(sat, obj_ant) si queremos que solo apunte a uno a la vez.

2.2. Calibrar Instrumento • Acción: calibrar (sat, inst) • Precondiciones: o instrumento (sat, inst, tipo) • Añadidos: o calibrado (sat, inst) • Borrados: o (ninguno)

2.3. Tomar Imagen • Acción: tomar_imagen(sat, inst, obj, resol) • Precondiciones: o apuntando (sat, obj) o instrumento (sat, inst, tipo_cam) o calibrado (sat, inst) o resolucion(inst, resol) o que la cámara de alta resolución puede hacer cualquier resol • Añadidos: o imagen_tomada(obj, resol) • Borrados: o calibrado (sat, inst) (pierde calibración tras el uso)

2.4. Tomar Medición • Acción: tomar_medicion(sat, inst, obj, tipo) • Precondiciones: o apuntando (sat, obj) o instrumento (sat, inst, tipo) o calibrado(sat, inst) • Añadidos: o medicion_tomada(obj, tipo) • Borrados: o calibrado(sat, inst)

3. Definición de ejemplo de estado inicial y meta.

3.1 Estado Inicial Representamos el estado inicial usando los predicados que definimos en la Parte 1:

Instrumentos y capacidades: instrumento(sat1, cam1, camara_alta) instrumento(sat1, medA, medidor_A) resolucion(cam1, alta)resolucion(cam1, media) resolucion(cam1, baja)

Posición inicial del satélite: apuntando(sat1, obj_inicial) % Los instrumentos están sin calibrar % (no incluimos calibrado(sat1, inst) porque es falso)

Tipos de objetivos: objetivo_tipo(obj1,

imagen_alta) objetivo_tipo(obj2, medicion_A) 3.2 Estado Meta Queremos que al final del día se cumpla que:

1. obj1 tenga imagen en alta resolución.
 2. obj2 tenga medición de tipo A. En STRIPS sería: imagen_tomada(obj1, alta) medicion_tomada(obj2, A)
- 3.3 Análisis del espacio de acciones Para lograr la meta, el planificador STRIPS deberá:
3. Apuntar el satélite al objetivo correspondiente.
 4. Calibrar el instrumento adecuado.
 5. Tomar la imagen o medición (lo cual borra la calibración).
 6. Repetir para el segundo objetivo.
 7. Construcción del plan paso a paso El objetivo es mostrar cómo STRIPS encadenaría las acciones desde el estado inicial hasta el estado meta.

4.1 Recordatorio de nuestro escenario • Estado inicial: o sat1 con cam1 (cámara alta) y medA (medidor A). o Satélite apuntando a obj_inicial. o Ningún instrumento calibrado. • Estado meta: imagen_tomada(obj1, alta) medicion_tomada(obj2, A) Acciones disponibles (resumen):

1. apuntar(sat, obj)
 2. calibrar (sat, inst)
 3. tomar_imagen(sat, inst, obj, resol)
 3. tomar_medicion(sat, inst, obj, tipo)
- Meta 1: Tomar imagen alta de obj1
4. Apuntar el satélite a obj1
apuntar (sat1, obj1) PRE: — ADD: apuntando (sat1, obj1)
DEL: apuntando (sat1, obj_inicial)
 5. Calibrar la cámara cam1
calibrar (sat1, cam1) PRE: instrumento (sat1, cam1, camara_alta) ADD: calibrado (sat1, cam1)
 6. Tomar la imagen de alta resolución
tomar_imagen(sat1, cam1, obj1, alta) PRE: apuntando (sat1, obj1), calibrado (sat1, cam1) ADD: imagen_tomada(obj1, alta) DEL: calibrado (sat1, cam1)
- Meta 2: Tomar medición tipo A de obj2
7. Apuntar el satélite a obj2
apuntar (sat1, obj2) PRE: — ADD: apuntando (sat1, obj2) DEL: apuntando (sat1, obj1)
 8. Calibrar el medidor medA
calibrar (sat1, medA) PRE: instrumento (sat1, medA, medidor_A) ADD: calibrado (sat1, medA)
 9. Tomar medición tipo A
tomar_medicion(sat1, medA, obj2, A) PRE: apuntando (sat1, obj2), calibrado (sat1, medA) ADD: medicion_tomada(obj2, A) DEL: calibrado(sat1, medA)

4.3 Plan Final en Orden STRIPS generaría el siguiente plan secuencial:

1. apuntar (sat1, obj1)
2. calibrar (sat1, cam1)
3. tomar_imagen(sat1, cam1, obj1, alta)

4. apuntar (sat1, obj2)
5. calibrar (sat1, medA)
6. tomar_medicion(sat1, medA, obj2, A) Al final del plan, se cumplen las metas:imagen_tomada(obj1, alta) medicion_tomada(obj2, A) y se ha usado correctamente la lógica de precondiciones, añadidos y borrados de STRIPS para que el satélite logre sus objetivos.

Conclusión U3 A2: Como observamos en esta actividad, se utilizó la metodología STRIPS la cual nos permite modelar un proceso de planificación para poder llevar a cabo una serie de actividades o metas, que al final es el resultado a que se requiere llegar. Se utilizó esta metodología de manera minuciosa para poder adaptarla a las necesidades de este problema de satélites, en donde lo que queríamos lograr es que un satélite pueda capturar una foto de algún objeto en el espacio y generar una medición genérica de ese mismo objeto o de otro (pudiese adaptarse a un meteorito, a un planeta o cualquier otro objeto del espacio). Para poder lograr el desarrollo de esta tarea, simplemente se siguió la metodología STRIPS, adaptándola paso a paso a cada proceso de este para poder llegar a la meta (identificar predicados, formalizar predicciones y precondiciones, generar instrumentos, etc.).

Unidad 4 Actividad 2.

Ejercicio 1. @prefix: <http://example.org/geo#> . @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> . @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> . @prefix owl: <http://www.w3.org/2002/07/owl#> . @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

Ontología
de Geografía (Ejercicio 1 de 5)

#####

:GeoOntology a owl:Ontology ; rdfs:label "Ontología de Geografía"@es ; rdfs:comment "Modelo básico para países, estados, ciudades, capitales y fronteras."@es .

Clases

:EntidadAdministrativa a owl:Class ; rdfs:label "Entidad Administrativa"@es . :Continente a owl:Class ; rdfs:subClassOf :EntidadAdministrativa ; rdfs:label "Continente"@es . :Pais a owl:Class ; rdfs:subClassOf :EntidadAdministrativa ; rdfs:label "País"@es . :Estado a owl:Class ; rdfs:subClassOf :EntidadAdministrativa ; rdfs:label "Estado/Provincia"@es . :Ciudad a owl:Class ; rdfs:label "Ciudad"@es . :FronteraTerrestre a owl:Class ; rdfs:label "Frontera Terrestre"@es . Disjuntas [] a owl:AllDisjointClasses ; owl:members (:Continente :Pais :Estado :Ciudad :FronteraTerrestre) . ##### Propiedades de objeto

:ubicadoEn a owl:ObjectProperty , owl:TransitiveProperty ; rdfs:label "ubicado en"@es ; rdfs:domain [a owl:Class ; owl:unionOf (:Pais :Estado :Ciudad)] ; rdfs:range :EntidadAdministrativa . :capitalDe a owl:ObjectProperty ; rdfs:label "capital de"@es ; rdfs:domain :Ciudad ; rdfs:range [a owl:Class ; owl:unionOf (:Pais :Estado)] . :tieneCapital a owl:ObjectProperty ; rdfs:label "tiene capital"@es ; rdfs:domain [a owl:Class ; owl:unionOf (:Pais :Estado)] ; rdfs:range :Ciudad ; owl:inverseOf :capitalDe . :limitaCon a owl:ObjectProperty , owl:SymmetricProperty , owl:IrreflexiveProperty ; rdfs:label "limita con"@es ; rdfs:domain :Pais ; rdfs:range :Pais . :fronteraCon a owl:ObjectProperty ; rdfs:label "frontera con"@es ; rdfs:domain :Pais ; rdfs:range :FronteraTerrestre .

Restricciones útiles

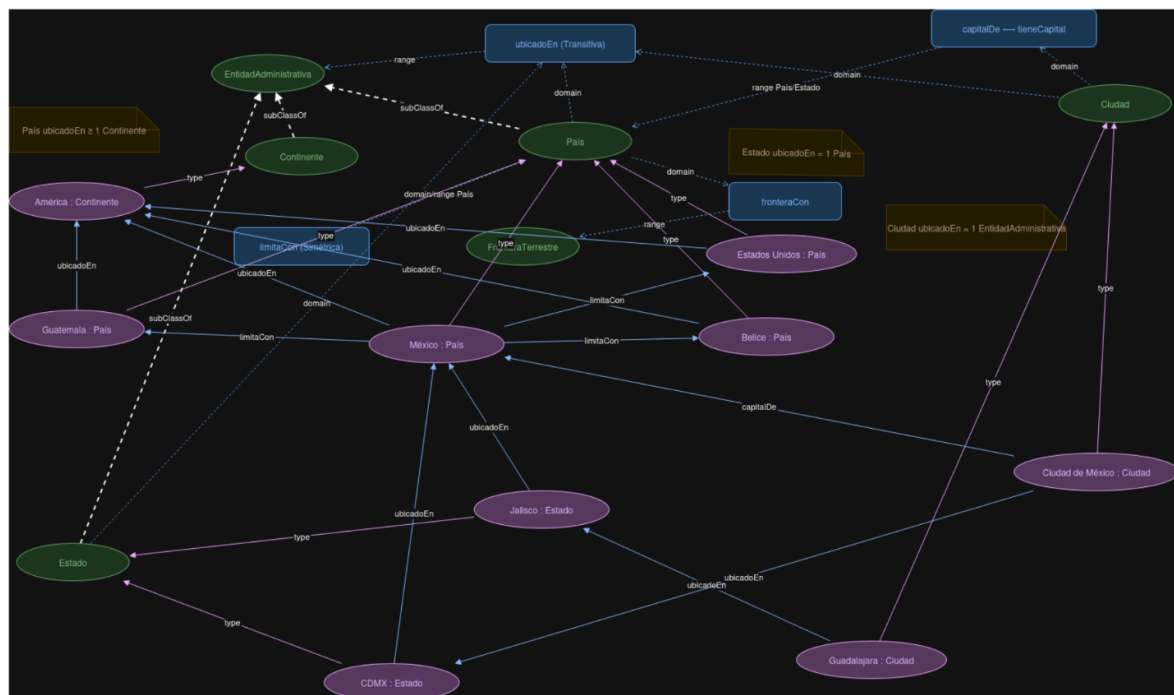
Toda Ciudad está ubicada exactamente en una Entidad Administrativa (país o estado) :Ciudad rdfs:subClassOf [a owl:Restriction ; owl:onProperty :ubicadoEn ; owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ; owl:onClass :EntidadAdministrativa] . Todo Estado está ubicado exactamente en un País :Estado rdfs:subClassOf [a owl:Restriction ; owl:onProperty :ubicadoEn ; owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ; owl:onClass :Pais] . Todo País está ubicado en al menos un Continente (maneja países transcontinentales) :Pais rdfs:subClassOf [a owl:Restriction ; owl:onProperty :ubicadoEn ; owl:someValuesFrom :Continente] .

Propiedades de datos

:poblacion a owl:DatatypeProperty ; rdfs:label "población"@es ; rdfs:domain [a owl:Class ; owl:unionOf (:Pais :Estado :Ciudad)] ; rdfs:range xsd:integer . :areaKm2 a owl:DatatypeProperty ; rdfs:label "área km²"@es ; rdfs:domain [a owl:Class ; owl:unionOf (:Pais :Estado :Ciudad)] ; rdfs:range xsd:decimal . :isoCode a owl:DatatypeProperty ; rdfs:label "código ISO"@es ; rdfs:domain :Pais ; rdfs:range xsd:string .

Individuos de ejemplo (México)

```
##### :America a owl:NamedIndividual , :Continente ; rdfs:label
"América"@es . :Mexico a owl:NamedIndividual , :Pais ; rdfs:label "México"@es ; isoCode
"MX" ; ubicadoEn :America ; poblacion 130000000 ; areaKm2 1964375 . :EstadosUnidos a
owl:NamedIndividual , :Pais ; rdfs:label "Estados Unidos"@es ; isoCode "US" ; ubicadoEn
:America . :Guatemala a owl:NamedIndividual , :Pais ; rdfs:label "Guatemala"@es ; isoCode
"GT" ; ubicadoEn :America . :Belize a owl:NamedIndividual , :Pais ; rdfs:label "Belize"@es ;
isoCode "BZ" ; ubicadoEn :America . :CDMX a owl:NamedIndividual , :Estado ; rdfs:label
"Ciudad de México (Entidad)"@es ; ubicadoEn :Mexico . :Jalisco a owl:NamedIndividual ,
:Estado ; rdfs:label "Jalisco"@es ; ubicadoEn :Mexico . :Guadalajara a owl:NamedIndividual ,
:Ciudad ; rdfs:label "Guadalajara"@es ; ubicadoEn :Jalisco ; poblacion 1500000 .
:CiudadDeMexico a owl:NamedIndividual , :Ciudad ; rdfs:label "Ciudad de México"@es ;
ubicadoEn :CDMX ; poblacion 9200000 ; capitalDe :Mexico . Fronteras y límites :MX_US a
owl:NamedIndividual , :FronteraTerrestre ; rdfs:label "Frontera México–EE.UU."@es .
:MX_GT a owl:NamedIndividual , :FronteraTerrestre ; rdfs:label "Frontera México–
Guatemala"@es . :MX_BZ a owl:NamedIndividual , :FronteraTerrestre ; rdfs:label "Frontera
México–Belize"@es . :Mexico :limitaCon :EstadosUnidos, :Guatemala, :Belize ; :fronteraCon
:MX_US, :MX_GT, :MX_BZ . :EstadosUnidos :limitaCon :Mexico . :Guatemala :limitaCon
:Mexico . :Belize :limitaCon :Mexico
```



Ejercicio 2.

@prefix : <http://example.org/biblio#> .

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

@prefix owl: <http://www.w3.org/2002/07/owl#> .

@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

#####

Ontología de Biblioteca (Ejercicio 2 de 5)

#####

:BiblioOntology a owl:Ontology ;

rdfs:label "Ontología de Biblioteca"@es ;

rdfs:comment "Modelo para libros, autores, editores, copias y préstamos."@es
.#####

Clases

#####

:Libro a owl:Class ; rdfs:label "Libro"@es .

:Copia a owl:Class ; rdfs:label "Copia"@es .

:Autor a owl:Class ; rdfs:label "Autor"@es .

:Editor a owl:Class ; rdfs:label "Editor/Editorial"@es .

:Biblioteca a owl:Class ; rdfs:label "Biblioteca"@es .

:Sucursal a owl:Class ; rdfs:label "Sucursal"@es ;

rdfs:subClassOf :Biblioteca .

:Usuario a owl:Class ; rdfs:label "Usuario"@es .

:Prestamo a owl:Class ; rdfs:label "Préstamo"@es .

Disjuntas (clases principales)

[] a owl:AllDisjointClasses ;

owl:members (:Libro :Copia :Autor :Editor :Biblioteca :Usuario :Prestamo)
.#####

Propiedades de objeto

#####

:escritoPor a owl:ObjectProperty ;

```
rdfs:label "escrito por"@es ;
rdfs:domain :Libro ;
rdfs:range :Autor .
:autorDe a owl:ObjectProperty ;
rdfs:label "autor de"@es ;
rdfs:domain :Autor ;
rdfs:range :Libro ;
owl:inverseOf :escritoPor .
:publicadoPor a owl:ObjectProperty ;
rdfs:label "publicado por"@es ;
rdfs:domain :Libro ; rdfs:range :Editor .
:editorDe a owl:ObjectProperty ;
rdfs:label "editor de"@es ;
rdfs:domain :Editor ;
rdfs:range :Libro ;
owl:inverseOf :publicadoPor .
:tieneCopia a owl:ObjectProperty ;
rdfs:label "tiene copia"@es ;
rdfs:domain :Libro ;
rdfs:range :Copia .
:esCopiaDe a owl:ObjectProperty , owl:FunctionalProperty ;
rdfs:label "es copia de"@es ;
rdfs:domain :Copia ;
rdfs:range :Libro ;
owl:inverseOf :tieneCopia . :perteneceA a owl:ObjectProperty ;
rdfs:label "pertenece a"@es ;
rdfs:domain :Copia ;
rdfs:range :Sucursal .
```

```

:ejemplar a owl:ObjectProperty ;
rdfs:label "ejemplar"@es ;
rdfs:domain :Prestamo ;
rdfs:range :Copia .

:usuario a owl:ObjectProperty ;
rdfs:label "usuario"@es ;
rdfs:domain :Prestamo ;
rdfs:range :Usuario .

#####

# Restricciones y reglas útiles#####

# Todo Libro tiene al menos un Autor
:Libro rdfs:subClassOf [ a owl:Restriction ;
owl:onProperty :escritoPor ;
owl:someValuesFrom :Autor ] .

# Toda Copia es copia de exactamente un Libro y pertenece a exactamente una Sucursal
:Copia rdfs:subClassOf
[ a owl:Restriction ; owl:onProperty :esCopiaDe ;
owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ; owl:onClass :Libro ],
[ a owl:Restriction ; owl:onProperty :perteneceA ;
owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ; owl:onClass :Sucursal ] .

# Todo Préstamo tiene exactamente un ejemplar y un usuario
:Prestamo rdfs:subClassOf
[ a owl:Restriction ; owl:onProperty :ejemplar ;
owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ; owl:onClass :Copia ],[ a
owl:Restriction ; owl:onProperty :usuario ;
owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ; owl:onClass :Usuario ] .

#####

# Propiedades de datos

```

#####

:titulo a owl:DatatypeProperty ;

rdfs:label "título"@es ;

rdfs:domain :Libro ;

rdfs:range xsd:string .

:isbn a owl:DatatypeProperty ;

rdfs:label "ISBN"@es ;

rdfs:domain :Libro ;

rdfs:range xsd:string .

:anioPublicacion a owl:DatatypeProperty ;rdfs:label "año de publicación"@es ;

rdfs:domain :Libro ;

rdfs:range xsd:gYear .

:nombre a owl:DatatypeProperty ;

rdfs:label "nombre"@es ;

rdfs:domain [a owl:Class ; owl:unionOf (:Autor :Editor :Sucursal :Usuario)] ;

rdfs:range xsd:string .

:numeroCopia a owl:DatatypeProperty ;

rdfs:label "número de copia"@es ;

rdfs:domain :Copia ;

rdfs:range xsd:string .

:fechaPrestamo a owl:DatatypeProperty ;

rdfs:label "fecha de préstamo"@es ;

rdfs:domain :Prestamo ;

rdfs:range xsd:date .:fechaDevolucionProgramada a owl:DatatypeProperty ;

rdfs:label "fecha de devolución programada"@es ;

rdfs:domain :Prestamo ;

rdfs:range xsd:date .

:fechaDevolucionReal a owl:DatatypeProperty ;

```

rdfs:label "fecha de devolución real"@es ;

rdfs:domain :Prestamo ;

rdfs:range xsd:date .

#####

# Individuos de ejemplo

#####

# Autores y editoriales

:AlicePerez a owl:NamedIndividual , :Autor ; :nombre "Alice Pérez" .

:BobLopez a owl:NamedIndividual , :Autor ; :nombre "Bob López" . :TecEditores a
owl:NamedIndividual , :Editor ; :nombre "TecEditores" .

# Sucursal

:CentralGDL a owl:NamedIndividual , :Sucursal ; :nombre "Biblioteca Central GDL" .

# Libros

:LibroSeguridad a owl:NamedIndividual , :Libro ;

:titulo "Fundamentos de Ciberseguridad" ;

:isbn "978-1-23456-789-0" ;

:anioPublicacion "2022"^^xsd:gYear ;

:escritoPor :AlicePerez , :BobLopez ;

:publicadoPor :TecEditores .

# Copias

:Copia001 a owl:NamedIndividual , :Copia ;

:numeroCopia "C-001" ;

:esCopiaDe :LibroSeguridad ;

:perteneceA :CentralGDL .# Usuario y préstamo

:LuisVargas a owl:NamedIndividual , :Usuario ; :nombre "Luis Vargas" .

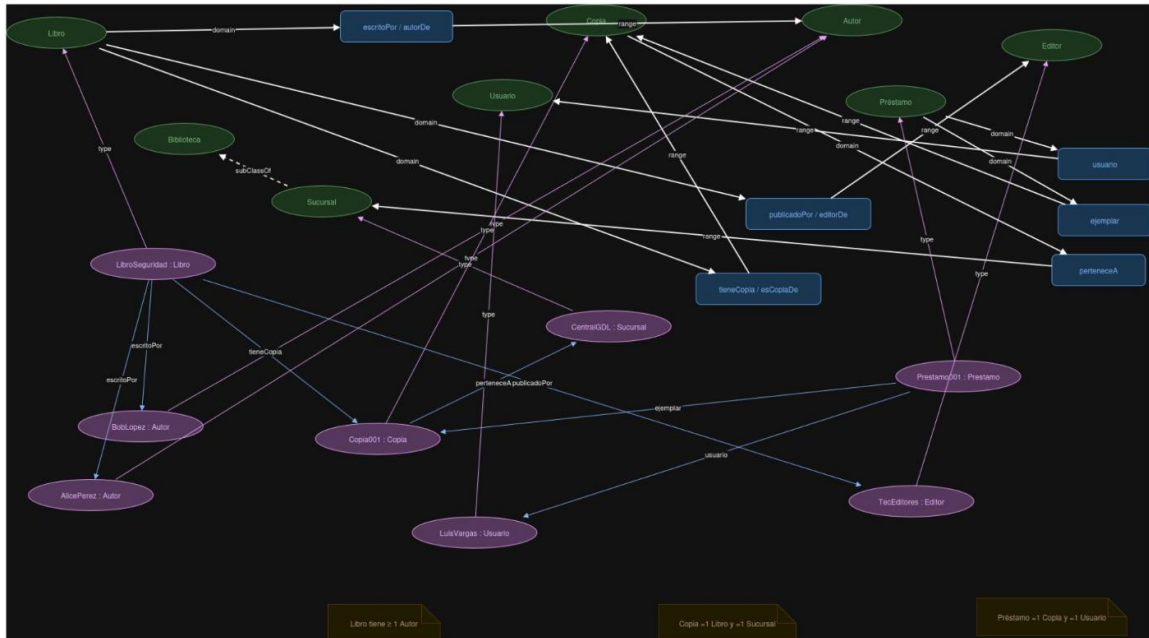
:Prestamo001 a owl:NamedIndividual , :Prestamo ;

:ejemplar :Copia001 ;

:usuario :LuisVargas ;

```

```
:fechaDevolucionProgramada "2025-08-21"^^xsd:date.
```



@prefix rdf: <<http://www.w3.org/1999/02/22-rdf-syntax-ns#>> .

@prefix owl: <<http://www.w3.org/2002/07/owl#>> .

#####

#####

```
:PersonasOntology a owl:Ontology ;
```


rdfs:label "Ontología Personas/Mamíferos"@es .

#####

Clases

#####

:Mamifero a owl:Class ;rdfs:label "Mamífero"@es .

:Persona a owl:Class ;

rdfs:label "Persona"@es ;

rdfs:subClassOf :Mamifero .

:PersonaFemenina a owl:Class ;

rdfs:label "Persona femenina"@es ;

rdfs:subClassOf :Persona .

:PersonaMasculina a owl:Class ;

rdfs:label "Persona masculina"@es ;

rdfs:subClassOf :Persona .

:Pierna a owl:Class ;

rdfs:label "Pierna"@es .

Disjuntas: femenina vs masculina[] a owl:AllDisjointClasses ;

owl:members (:PersonaFemenina :PersonaMasculina) .

#####

Propiedades de objeto

#####

:tienePierna a owl:ObjectProperty ;

rdfs:label "tiene pierna"@es ;

rdfs:domain :Persona ;

rdfs:range :Pierna .

:tieneMadre a owl:ObjectProperty ;

rdfs:label "tiene madre"@es ;

rdfs:domain :Persona ;

```

rdfs:range :PersonaFemenina .

:hermanaDe a owl:ObjectProperty ;rdfs:label "hermana de"@es ;
rdfs:domain :PersonaFemenina ;
rdfs:range :Persona .

:tieneHermana a owl:ObjectProperty ;
rdfs:label "tiene hermana"@es ;
rdfs:domain :Persona ;
rdfs:range :PersonaFemenina ;
owl:inverseOf :hermanaDe .

#####

# Restricciones sobre clases

#####

# Toda Persona tiene exactamente 2 piernas

:Persona rdfs:subClassOf [
a owl:Restriction ;
owl:onProperty :tienePierna ;owl:qualifiedCardinality "2"^^xsd:nonNegativeInteger ;
owl:onClass :Pierna
].

# Toda Persona tiene al menos una madre (de tipo PersonaFemenina)

:Persona rdfs:subClassOf [
a owl:Restriction ;
owl:onProperty :tieneMadre ;
owl:someValuesFrom :PersonaFemenina
].

#####

# Propiedades de datos

#####

:nombre a owl:DatatypeProperty ;

```

```

rdfs:label "nombre"@es ;

rdfs:domain :Persona ;rdfs:range xsd:string .

#####

# Individuos de ejemplo

#####

:Mary a owl:NamedIndividual , :PersonaFemenina ;

:nombre "Mary" .

:John a owl:NamedIndividual , :PersonaMasculina ;

:nombre "John" .

# Relación de hermandad (Mary es hermana de John)

:Mary :hermanaDe :John .

```

Ejercicio 4.@prefix : <http://example.org/veg#> .

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

@prefix rdfs:<http://www.w3.org/2000/01/rdf-schema#> .

@prefix owl: <http://www.w3.org/2002/07/owl#> .

@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

#####

Ontología 4/5

#####

:VegOntology a owl:Ontology ;

rdfs:label "Ontología Ejercicio 4"@es .

#####

Clases

#####:Persona owl:Class .

:Mana owl:Class ; rdfs:subClassOf :Person , :Adult , :Male .

:Adulta owl:Class .

```

:Malea owl:Class .

:Animala owl:Class .

:Cata owl:Class ; rdfs:subClassOf :Animal .

:Cowa owl:Class ; rdfs:subClassOf :Animal .

:Sheepa owl:Class ; rdfs:subClassOf :Animal ,
[ a owl:Restriction ; owl:onProperty :eats ; owl:allValuesFrom :Grass ] .

:Grass
a owl:Class .

# Definidas por propiedades

:CatLiker a owl:Class ;

owl:equivalentClass [a owl:Class ;

owl:intersectionOf (

:Person

[ a owl:Restriction ; owl:onProperty :likes ; owl:someValuesFrom :Cat ]

)

].

:Vegetarian a owl:Class ;

rdfs:subClassOf :Person ,

[ a owl:Restriction ; owl:onProperty :eats ;

owl:allValuesFrom [ a owl:Class ; owl:complementOf :Animal ] ] .

#####

# Propiedades de objeto

#####:hasPet a owl:ObjectProperty ; rdfs:domain :Person ;
rdfs:range :Animal .

:likes a owl:ObjectProperty .

:eats a owl:ObjectProperty .

#####

# Individuos (solo los del diagrama)

```

#####

:Fred a owl:NamedIndividual , :Person ;

:hasPet :Tibbs .

:Tibbs a owl:NamedIndividual , :Cat .

Ejercicio 5.

@prefix : <http://example.org/pets2#> .

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

@prefix rdfs:<http://www.w3.org/2000/01/rdf-schema#> .

@prefix owl: <http://www.w3.org/2002/07/owl#> .

@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .:Onto5 a owl:Ontology ;

rdfs:label "Ejercicio 5 (owns / hasPet / petOf / sameAs)"@es .

Clases

:Animal a owl:Class .

:Mammal a owl:Class ; rdfs:subClassOf :Animal .

:Dog

a owl:Class ; rdfs:subClassOf :Mammal .

:Human a owl:Class .

(opcionales del dibujo: conteos)

:Eyes a owl:DatatypeProperty .

:Legs a owl:DatatypeProperty .

Propiedades de objeto

:owns a owl:ObjectProperty ; rdfs:label "owns"@en ;

rdfs:domain :Human ;

rdfs:range owl:Thing .

:hasPet a owl:ObjectProperty ;

rdfs:label "hasPet"@en ;

```

rdfs:domain :Human ;
rdfs:range :Mammal ;
rdfs:subPropertyOf :owns .
:petOf a owl:ObjectProperty ;
rdfs:label "petOf"@en ;
rdfs:domain :Mammal ;
rdfs:range :Human ;

owl:inverseOf :hasPet .##### Propiedades de datos #####

:eyes a owl:DatatypeProperty ;
rdfs:label "eyes"@en ;
rdfs:domain :Animal ;
rdfs:range xsd:integer .

:legs a owl:DatatypeProperty ;
rdfs:label "legs"@en ;
rdfs:domain :Mammal ;
rdfs:range xsd:integer .

##### Restricciones de cardinalidad #####

# Todo Animal tiene exactamente 2 ojos

:Animal rdfs:subClassOf [a owl:Restriction ;
owl:onProperty :eyes ;
owl:hasValue 2
].

# Todo Mamífero tiene exactamente 4 patas

:Mammal rdfs:subClassOf [
a owl:Restriction ;
owl:onProperty :legs ;
owl:hasValue 4
].

```

Individuos

Humanos

:Jans a owl:NamedIndividual , :Human .

:MrAsman a owl:NamedIndividual , :Human .# El diagrama marca que Jans y MrAsman son el mismo individuo

:Jans owl:sameAs :MrAsman .

Mascota (mamífero)

:Robbie a owl:NamedIndividual , :Dog .

Relaciones (del diagrama)

Robbie es mascota de Jans → petOf

:Robbie :petOf :Jans .

Por inversa, el razonador infiere :Jans :hasPet :Robbie,

pero si quieres que quede explícito:

:Jans :hasPet :Robbie .

(Opcional) También es válido usar owns; hasPet $\sqsubseteq$ owns,

:Jans :owns :Robbie

Conclusión Unidad 4 Actividad 2: A manera de conclusión podemos afirmar que se crearon las ontologías de manera satisfactoria, pues como podemos ver se solucionan los ejercicios que se nos presentan, para poder resolverlos de manera correcta, fue un poco complicado por la cantidad de variables, precondiciones y relaciones entre las diferentes identidades, se tuvo que realizar varias veces cada uno hasta que “hiciese sentido “y fuese lógicamente adaptado a el problema, creando las clases, las disjuntas , además se consultó un libro adicional de referencia el cual citaré en la sección de referencias bibliográficas.

Unidad 5 Actividad 2.

Ejercicio 1. Datos del ejercicio

- $P(+c|+a,+b)=0.99$ $P(+c|-a,+b)=0.9$ $P(+c|+a,-b)=0.8$ $P(+c|-a,-b)=0.001$
- $P(+d|+c)=0.99$ $P(+d|-c)=0.9$
- Supuesto: $P(d|c,a,b)=P(d|c)$ (separación direccional).

(a) Probabilidad de A dado C y B conocido Fórmula (Bayes): $P(+a | +c, -b) = [P(+c | +a, -b) \cdot P(+a)] / \{P(+c | +a, -b) \cdot P(+a) + P(+c | -a, -b) \cdot P(-a)\}$ Constantes: $P(+c | +a, -b) = 0.8$ $P(+c | -a, -b) = 0.001$ $P(+a) = 0.01$, $P(-a) = 0.99$ Cálculo: $P(+a | +c, -b) = [0.80 \cdot 0.01] / [0.80 \cdot 0.01 + 0.001 \cdot 0.99]$ $P(+a | +c, -b) \approx 0.889877642$

(b) Probabilidad de C dado D Paso 1: marginal de C $P(+c) = \sum_{a,b} P(+c | a,b) \cdot P(a) \cdot P(b) = 0.01 \cdot (0.99 \cdot 0.006 + 0.80 \cdot 0.994) + 0.99 \cdot (0.90 \cdot 0.006 + 0.001 \cdot 0.994)$

$P(+c) \approx 0.01434146$ Paso 2 : Bayes con D $P(+c \mid +d) = [P(+d \mid +c) \cdot P(+c)] / \{P(+d \mid +c) \cdot P(+c) + P(+d \mid -c) \cdot [1 - P(+c)]\}$ Constantes: $P(+d \mid +c) = 0.99$ $P(+d \mid -c) = 0.90$ Cálculo: $P(+c \mid +d) = [0.99 \cdot 0.01434146] / [0.99 \cdot 0.01434146 + 0.90 \cdot (1 - 0.01434146)]$ $P(+c \mid +d) \approx 0.015753014$ (c) Probabilidad posterior de C con evidencia de D y A, B conocidos Fórmula (con separación direccional): $P(+c \mid +d, +a, b) = [P(+d \mid +c) \cdot P(+c \mid +a, b)] / \{P(+d \mid +c) \cdot P(+c \mid +a, b) + P(+d \mid -c) \cdot [1 - P(+c \mid +a, b)]\}$ Constantes usadas: $P(+d \mid +c) = 0.99$ $P(+d \mid -c) = 0.90$ $P(+c \mid +a, +b) = 0.99$ $P(+c \mid +a, -b) = 0.80$ Caso 1: $b = +b$ $P(+c \mid +d, +a, +b) = [0.99 \cdot 0.99] / [0.99 \cdot 0.99 + 0.90 \cdot (1 - 0.99)]$ $P(+c \mid +d, +a, +b) \approx 0.990900819$ Caso 2: $b = -b$ $P(+c \mid +d, +a, -b) = [0.99 \cdot 0.80] / [0.99 \cdot 0.80 + 0.90 \cdot (1 - 0.80)]$ $P(+c \mid +d, +a, -b) \approx 0.814814815$

Ejercicio 2. Ejercicio 2 – Red bayesiana con A (edad), B (ingresos), C (nivel de vida), D (salud) Estructura (DAG): $A \rightarrow B \rightarrow C$ y $A \rightarrow D$ Dominios: • $A \in \{a1=joven, a2=medio, a3=viejo\}$ • $B \in \{b1=alto, b2=bajo\}$ • $C \in \{c1=bueno, c2=malo\}$ • $D \in \{d1=buena, d2=mala\}$ Parámetros (del enunciado): • $P(a1) = 0.3$, $P(a3) = 0.2 \Rightarrow P(a2) = 0.5$ • $P(b1|a1) = 0.1$, $P(b1|a2) = 0.3$, $P(b1|a3) = 0.2 \Rightarrow P(b2|a1) = 0.9$, $P(b2|a2) = 0.7$, $P(b2|a3) = 0.8$ • $P(c1|b1) = 0.9$, $P(c1|b2) = 0.1 \Rightarrow P(c2|b1) = 0.1$, $P(c2|b2) = 0.9$ • $P(d1|a1) = 0.9$, $P(d1|a2) = 0.7$, $P(d1|a3) = 0.3 \Rightarrow P(d2|a1) = 1 - P(d1|a1)$.

=====
 ==== (a) Actualizar probabilidades sabiendo $B=b2$ (ingresos bajos). Tipo de razonamiento: Causal/pronóstico (de la causa B a sus descendientes y actualización de A por la relación $A \rightarrow B$). Paso 1. Prior de B: $P(b1) = 0.1 \cdot 0.3 + 0.3 \cdot 0.5 + 0.2 \cdot 0.2 = 0.22$ $P(b2) = 1 - 0.22 = 0.78$ Paso 2. Posterior de la edad dado $B=b2$ (Bayes): $P(a1 \mid b2) = [P(b2 \mid a1) \cdot P(a1)] / \sum_j P(b2 \mid aj) \cdot P(aj)$ Resultados: $P(a1 \mid b2) = (0.9 \cdot 0.3) / 0.78 \approx 0.346$ $P(a2 \mid b2) = (0.7 \cdot 0.5) / 0.78 \approx 0.449$ $P(a3 \mid b2) = (0.8 \cdot 0.2) / 0.78 \approx 0.205$ Paso 3. Nivel de vida con $B=b2$ (directo de la CPT $C \leftarrow B$): $P(c1 \mid b2) = 0.1$ $P(c2 \mid b2) = 0.9$ Paso 4. Salud con $B=b2$ (marginalizando A): $P(d1 \mid b2) = \sum_i P(d1 \mid a1) \cdot P(a1 \mid b2) = 0.9 \cdot 0.346 + 0.7 \cdot 0.449 + 0.3 \cdot 0.205 \approx 0.687$ $P(d2 \mid b2) = 1 - 0.687 \approx 0.313$ Resumen (a): • $P(a1 \mid b2) \approx 0.346$, $P(a2 \mid b2) \approx 0.449$, $P(a3 \mid b2) \approx 0.205$ • $P(c1 \mid b2) = 0.1$, $P(c2 \mid b2) = 0.9$ • $P(d1 \mid b2) \approx 0.687$, $P(d2 \mid b2) \approx 0.313$ Comentario breve: al observar ingresos bajos ($b2$), aumenta la probabilidad de edad media ($a2$) y baja la de ingresos altos; la salud esperada queda alrededor de 0.687 para $d1$.

=====

==== (b) Variables independientes y correlacionadas. Mini-red con {peso, tensión arterial, riesgo de infarto}. Con el DAG $A \rightarrow B \rightarrow C$ y $A \rightarrow D$: Correlaciones marginales (no independientes sin condicionar):

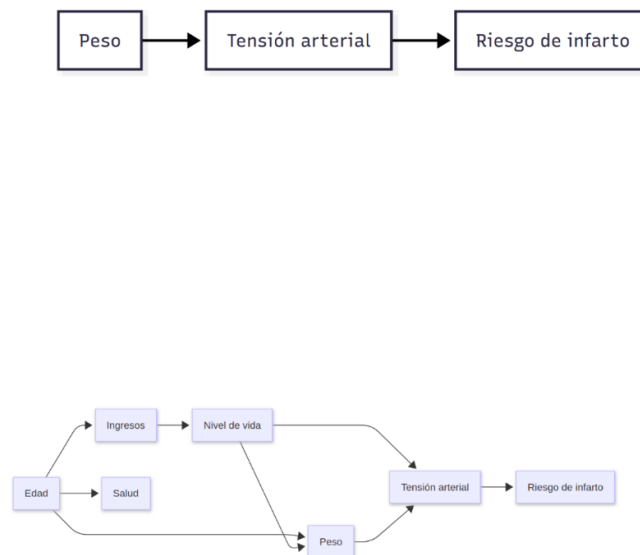
- (A, B), (A, C), (A, D) están correlacionadas (A es ancestro).
- (B, D) están correlacionadas por la causa común A.
- (C, D) están correlacionadas por el camino $C \leftarrow B \leftarrow A \rightarrow D$.
- (B, C) no son independientes porque C depende de B.

Independencias condicionales (por d-separación):

- $B \perp D | A$
- $C \perp A | B$
- $C \perp D | B$

Mini-red cardiovascular propuesta: $\text{peso} \rightarrow \text{tensión} \rightarrow \text{riesgo_de_infarto}$

Propiedad clave: $\text{riesgo_de_infarto} \perp \text{peso} \mid \text{tensión}$



Conclusión Unidad 5 Actividad 2: Como pudimos ver, se resolvieron los 3 ejercicios propuestos, para poder resolverlos de manera correcta, se necesitó investigar todas las

fórmulas de bayes para poder realizar las inferencias, practicando con los Dominios, y adaptando las fórmulas para poder escribirlas matemáticamente, y después adaptarlas al formato PDF/Word.

Conclusión General.

Como conclusión general podemos decir que se mostraron de manera correcta las actividades que se realizaron en esta materia, fue interesante ver y demostrar el potencial que estos temas tienen para el área de ciberseguridad y para IA especialmente ya que como sabemos la web semántica, la planificación son temas fundamentales para el correcto funcionamiento de la IA, su gran ingesta de datos, y su procesado de datos, además, como sabemos, el tema de lógica clásica, difusa son temas base para como “piensa” y procesa la IA, es algo muy importante de denotar porque como sabemos incluso los equipos de detección y respuesta como los EDR, XDR, IDS/IPS usan la lógica clásica para poder determinar intrusiones en redes corporativas, plataformas de código online compartido como GitHub, por lo que ambas materias, IA y ciberseguridad se acercan cada vez más.

Es además importante denotar que; existen temas importantes dentro de esta área como el de redes bayesianas o el de Web Semántica, son temas en continuo desarrollo y por ende en continuo cambio. Es importante decir que son temas que, en 10 años por decir, tengamos mucha más y mejor información de la que tenemos hoy en día, pero hoy a hoy, esta es una buena introducción para este campo del conocimiento fusionando IA/Ciberseguridad/Big Data.

Referencias Bibliográficas.

1. *ONTOLOGÍAS* (2020.^a ed., Vol. 1). (2020). [Internet]. Blas Torregrosa García. <https://openaccess.uoc.edu/server/api/core/bitstreams/5c06f02d-67ee-47d0-8ffa-ad709c24be51/content>
2. Raju, H. (2024, 18 noviembre). *Exploring Clingo: A Comprehensive Guide*. Medium. <https://medium.com/design-bootcamp/exploring-clingo-a-comprehensive-guide-5e577f163036>
3. Cephei. (2018, 21 octubre). Installing and Using Clingo on Windows. *Spaceship Parkling*. https://spaceshipparking.blogspot.com/2018/10/installing-and-using-clingo-on-windows_21.html
4. GeeksforGeeks. (2025, 23 julio). *STRIPS in AI*. GeeksforGeeks. https://www-geeksforgeeks-org.translate.goog/artificial-intelligence/strips-in-ai/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
5. Marin, R., & Jose, P. (s. f.). *Inteligencia artificial. Técnicas, métodos y aplicaciones*.
6. Planificación en Inteligencia Artificial. Agentes Planificadores. (2021). *UDB*, 21(003). https://www.udb.edu.sv/udb_files/recursos_guias/informatica-ingenieria/sistemas-expertos-e-inteligencia-artificial/2020/i/guia-11.pdf
7. Redes Bayesianas. (2018). En *INAOE*. Luis Enrique Sucar. Recuperado 18 de agosto de 2025, de <https://ccc.inaoep.mx/~esucar/Clases-mgp/caprb.pdf>